
RELAYSPEC: REUSING A FROZEN SPECULATIVE PROPOSER ON A NEW TARGET MODEL WITH ONE LINEAR MAP

Anonymous authors

Paper under double-blind review

ABSTRACT

Speculative decoding accelerates generation by letting a small proposer guess several tokens ahead and letting the target model verify all of them in one pass. The strongest proposers, such as DFlash and EAGLE-3, condition on hidden states read out of the exact model they were trained with, which we call the source model. That coupling is the problem: serving a different target model with the same proposer normally means training a new proposer, which DFlash reports as 800,000 examples over 6 epochs, or keeping the source model resident forever purely to rebuild the proposer’s input. We show that a single linear map removes the coupling. The map is fitted on 4,096 unlabeled examples in under two minutes, reads five hidden states the new target already computes while verifying, and emits the tensor the frozen proposer expects, after which the source model can be deleted entirely. On MATH-500, against plain autoregressive decoding measured in the same paired run, RelaySpec reaches 5.02x and 5.14x with DFlash on Qwen3-8B and Qwen3-14B, and 2.40x and 2.62x with EAGLE-3, while retaining 89 to 90 percent of the throughput a proposer trained for that exact target reaches and cutting peak GPU memory by 20 to 31 percent. The same frozen proposer moves further than its own training pair: onto an independently fine-tuned descendant with no refitting (4.92x), onto a larger target through an adapter (4.12x), into a different family and onto a target smaller than its own source (2.45x), and across a tokenizer boundary (2.26x). Every speedup we report is measured against plain autoregressive decoding, paired request by request, with bootstrap confidence intervals.

1 INTRODUCTION

A speculative decoder pairs a small proposer with a large target model. The proposer guesses several tokens, the target checks them all in one forward pass, and every token the target disagrees with is thrown away. Output text is unchanged, so the only question is speed.

The strongest proposers today do not read tokens alone. DFlash (Chen et al., 2026) and EAGLE-3 (Li et al., 2025) also read internal hidden states from the model they were trained against. This makes them accurate, and it also ties them to that one model. Serving a different target model with the same proposer means either training a new proposer, or running the old source model on every decoding step purely to rebuild the proposer’s input, even though that source model never approves a single token.

Training a new proposer is expensive. DFlash trains its drafter on about 800,000 examples for 6 epochs, and EAGLE-3 uses roughly 532,000 examples. Neither reports GPU-hours, but both are separate training runs per target model.

We ask whether the tensor the proposer needs can simply be rebuilt from the new target, and answer yes for a useful range of cases. RelaySpec fits one linear map that reads five hidden states the target model already produces during ordinary verification, and outputs the tensor the frozen proposer expects. Fitting uses 4,096 unlabeled examples and takes 86 to 119 seconds on four GPUs. Nothing inside the proposer changes. After fitting, the source model can be deleted. Figure 1 shows the change.

054
055
056
057
058
059
060
061
062
063
064
065
066
067
068
069
070
071
072
073
074
075
076
077
078
079
080
081
082
083
084
085
086
087
088
089
090
091
092
093
094
095
096
097
098
099
100
101
102
103
104
105
106
107

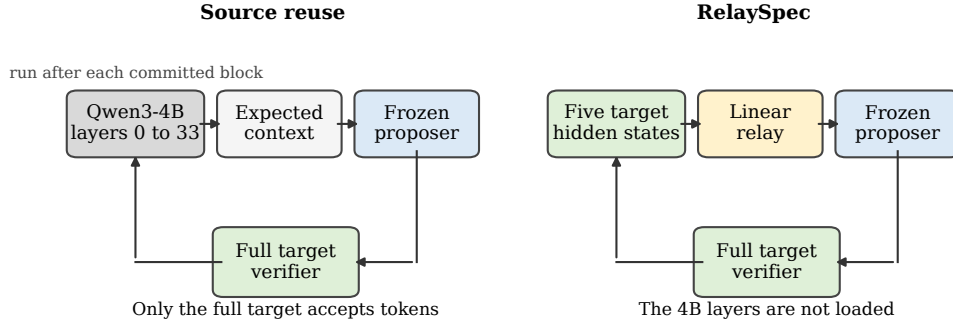


Figure 1: Left: reusing a proposer today. The source model runs every step just to rebuild the proposer’s input. Right: RelaySpec. One linear map rebuilds the same input from hidden states the target already computed, and the source model is gone. The target model verifies every token in both cases.

We make four contributions.

A two-minute replacement for retraining a proposer. One linear map, fitted on 4,096 unlabeled examples, delivers 5.02x to 5.14x over plain autoregressive decoding with DFlash and 2.40x to 2.62x with EAGLE-3, retaining 89 to 90 percent of what a proposer trained for that exact target reaches, at roughly three orders of magnitude less adaptation data (Section 6).

A rule that says in advance when it will pay off. A short equation over measured timing and accepted tokens per cycle predicts whether removing the source model helps or hurts. It gets the direction right in all 16 additional task settings we test (Section 4).

One frozen proposer, four kinds of target change. The same map transfers onto an independently fine-tuned descendant of the target with no refitting, onto a larger target through an adapter over a frozen decoder, into a different model family and onto a smaller target than the proposer’s own source, and across a tokenizer boundary. All four remain well above plain autoregressive decoding (Section 7).

Design evidence that a plain linear map is the right hypothesis. Against a closed-form solution of the same objective and a nonlinear map at the same fitting budget, the linear map fitted by gradient descent wins clearly, and fit quality is still improving at 8,192 examples (Appendix F).

2 WHAT ONE DECODING CYCLE DOES

At each cycle the proposer produces a block of candidate tokens. The target model runs once over that block and produces, at every position, the token it would have generated itself. We keep the longest starting run where the candidates match the target’s own choices, then append the target’s next token. Everything after the first disagreement is discarded. Because every committed token is a token the target itself produced, output text does not depend on proposal quality. Proposal quality only changes how many cycles are needed.

A feature-conditioned proposer needs one extra input: a continuous context tensor built from the source model’s hidden states. RelaySpec produces that tensor from the target instead. Figure 2 shows the loop.

Four structural conditions must hold before the swap is even possible (Table 1). They say whether it can be done, not whether it is fast.

3 METHOD

The slot we are replacing is already one linear layer. Both released proposers read their source model the same way. They concatenate five source hidden states into z_t^S , pass them through one

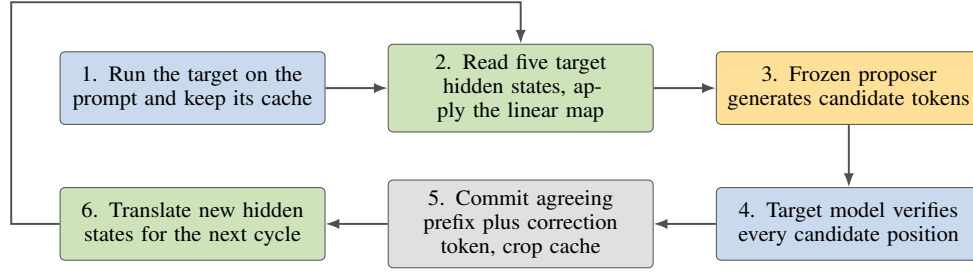


Figure 2: One RelaySpec decoding cycle. Green boxes read or translate target hidden states. Blue boxes are the full target model, the only component that can accept a token. Step 6 loops back into step 2.

Table 1: Conditions a proposer and target pair must satisfy for RelaySpec to apply. Section 4 covers whether it is also faster.

Condition	What this means in practice
Same token meaning	Proposer and target use the same token IDs, including special tokens. Section 7 measures what happens when they do not.
Separate continuous input	The proposer takes a continuous context tensor as its own input. This is the tensor the map learns to produce.
Target exposes hidden states	Ordinary verification already computes the target’s hidden states, so they only need to be read out.
Verification is independent	Target verification and cache updates never depend on the translated tensor, so the target stays the only component that can accept a token.

linear fusion layer F that the proposer ships with, and normalize the result before any proposer block sees it: the conditioning tensor is $\text{norm}(Fz_t^S)$. RelaySpec deletes F and puts a map R in its place, reading the target’s own five hidden states z_t^T instead, so the tensor becomes $\text{norm}(Rz_t^T)$. Nothing downstream of that normalization changes by a single weight.

This is the reason a linear map is the right hypothesis class rather than a convenient one. We are not approximating the source model’s computation. We are refitting the one linear layer the proposer already used to read a model, now pointed at a different model. Linear is the class the proposer’s own authors chose for this exact slot, and staying inside it is the minimal edit that leaves the rest of the frozen proposer in the distribution it was trained on. An MLP at the same fitting budget does not beat it (Appendix F).

What is fitted, exactly. R is a single weight matrix with no bias, applied after an RMSNorm over the concatenated input. At Qwen3-8B the target hidden size is 4,096 and five taps give an input width of 20,480, so R is $20,480 \times 2,560$, or 52.4 million parameters. At Qwen3-14B the hidden size is 5,120 and R is $25,600 \times 2,560$, or 65.5 million. The output width 2,560 is fixed by the frozen proposer and never varies. Taps are layers [1, 9, 17, 25, 33] of 36 at 8B and [1, 10, 19, 28, 37] of 40 at 14B, the same relative depths the proposer’s own source taps sit at. Fitting is AdamW at learning rate 6×10^{-4} for 1,024 steps over 4,096 unlabeled prompts truncated to 192 tokens, one pass, four GPUs, 86 to 119 seconds wall clock. The proposer, the target and the source model are all frozen throughout, R is the only thing that moves, and no answer labels are used at any point.

Why the loss is normalized by the target’s own energy. Training minimizes

$$\mathcal{L} = \frac{\|Rz_t^T - c_t^S\|^2}{\|c_t^S\|^2 + \epsilon}, \quad (1)$$

per position, where $c_t^S = Fz_t^S$ is what the original fusion layer would have produced. The denominator is not a convenience. The normalization that immediately follows this slot removes the overall magnitude of whatever we hand it, so the proposer can only ever see the direction of Rz_t^T . An unnormalized squared error would spend the map’s capacity on a scale the proposer discards,

Algorithm 1 RelaySpec decoding. Only line 8 can commit a token.

```

1: Run target on the prompt, keep its cache and hidden states
2:  $c \leftarrow R$ (five target hidden states)
3: while output is not finished do
4:    $B \leftarrow$  proposer generates a candidate block, conditioned on  $c$ 
5:   Run target once over  $B$ , keeping its cache
6:    $P \leftarrow$  token the target itself would emit at each position of  $B$ 
7:    $k \leftarrow$  length of the longest starting run where  $B$  agrees with  $P$ 
8:   Commit those  $k$  tokens, then commit the target’s own next token
9:   Crop target cache to the committed length
10:   $c \leftarrow R$ (target hidden states at the committed positions)
11: end while

```

and would let the few positions with the largest activations dominate the fit. Dividing by $\|c_t^S\|^2$ makes every position count equally and makes the objective measure the only thing the proposer can distinguish. The same argument is why R carries no bias term.

At inference the source model is gone. Algorithm 1 gives the loop actually executed.

The map only changes what the proposer sees. It never approves a token. A poor map therefore causes more cycles, never different text.

Proposition 1 *Under greedy decoding and exact arithmetic, RelaySpec produces the same output as running the target model alone.*

Every committed token is taken from the target’s own greedy output at that position, given a prefix the target itself produced. Induction over committed positions gives the result. In practice BF16 kernels for a block of positions and for one position can differ slightly near ties. We therefore report exact text agreement as a measurement: 500/500 of MATH-500 completions for DFlash at 8B, and 496/500 for EAGLE-3, with official accuracy unchanged in every case.

4 WHEN IS THIS WORTH DOING?

Removing the source model only helps if candidates stay good enough. Split total source-reuse time, normalized to one, into three parts,

$$1 = p_C + p_S + p_O, \quad (2)$$

where p_C scales with the number of cycles, p_S is the removable source-model share, and p_O is everything else. Let p_R be the map’s own time share, and a_S, a_R the tokens committed per cycle before and after. Since cycle count is roughly inversely proportional to tokens per cycle,

$$\frac{L_R}{L_S} = \frac{a_S}{a_R} p_C + p_O + p_R, \quad \text{so RelaySpec is faster exactly when} \quad \frac{a_R}{a_S} > \frac{p_C}{1 - p_O - p_R}. \quad (3)$$

The right side is the smallest fraction of the original accepted-token rate the map must keep, and every quantity in it is measurable on a short sample of real requests before committing to either configuration. This is the one place in the paper where the comparison is against source reuse rather than plain autoregressive decoding, because the question it answers is specifically whether to unload the source model, not how fast either option is: both sides of Equation 3 are speculative decoding, so the plain-AR floor cancels.

The rule also explains a size effect we measure rather than merely fitting it. A fixed 4B source model is a smaller share of a 14B target’s total work than of an 8B target’s, so p_S shrinks as the target grows, the removable term is smaller, and the map has to preserve a higher fraction of the accepted-token rate to break even. That is exactly the ordering Table 2 shows: relay over source reuse falls from 1.53x at 8B to 1.27x at 14B with DFlash, and from 1.28x to 1.10x with EAGLE-3, while the speedup over plain autoregressive decoding rises.

5 SETUP

The proposers are released DFlash and EAGLE-3 checkpoints trained against Qwen3-4B. Targets are Qwen3-8B and Qwen3-14B (Yang et al., 2025). We fit one map per proposer and target pair on 4,096 math prompts, using no answer labels. Evaluation covers MATH-500 (Hendrycks et al., 2021), GSM8K (Cobbe et al., 2021), HumanEval (Chen et al., 2021), MBPP (Austin et al., 2021) scored with EvalPlus (Liu et al., 2023), and MT-Bench (Zheng et al., 2023), which we time but do not score. All runs use greedy decoding, one request at a time, BF16, on four RTX 6000 Ada GPUs. Fitting prompts and evaluation prompts are disjoint, and Appendix G reports the overlap audit.

We compare against three baselines, and define each precisely because the paper reports ratios against more than one of them.

Plain autoregressive decoding (native AR). The target model alone, one token per forward pass, no proposer and no source model involved. This is the deployment a user reaches for absent a target-specific proposer or a source model worth keeping resident, and the comparison every reader can act on unconditionally. Every main-table and transfer speedup in this paper is measured against native AR in the same paired run as the other methods compared.

Optimized source reuse. The alternative a user has today without RelaySpec: keep the source model the proposer was trained against resident alongside the target, running it on every newly committed token purely to reconstruct the tensor the proposer needs, truncated after the last layer the proposer reads (never re-run on a rejected candidate suffix). This isolates what RelaySpec specifically buys: the same frozen proposer and accepted-token behavior, with the source model deleted instead of kept resident. We report it as a secondary ratio alongside native AR, never as the primary claim.

Target-specific proposer. A proposer trained or released specifically for the deployed target, where such a checkpoint exists publicly. This is the ceiling: what full retraining, on hundreds of thousands of examples, would buy over RelaySpec’s two-minute linear fit.

6 DOES IT WORK, AND WHAT DOES IT COST?

Table 2 gives the main MATH-500 result. Every method in a row, including plain autoregressive decoding, ran inside one benchmark process on the same 500 prompts, so each ratio is paired request by request rather than compared across executions.

Table 2: MATH-500, 500 prompts, greedy decoding. Every speedup is a paired bootstrap ratio of total request time against plain autoregressive decoding measured in the same run. RelaySpec is the frozen proposer driven by one linear map, with the source model unloaded. Accepted tokens per cycle are reported for source reuse (S) and RelaySpec (R).

Proposer	Target	Speedup over plain AR		Relay over source reuse	Accepted tokens per cycle S/R
		Source reuse	RelaySpec [95% CI]		
DFlash	8B	3.28x	5.02x [4.87, 5.17]	1.53x	7.66/6.98
DFlash	14B	4.06x	5.14x [4.98, 5.30]	1.27x	7.44/6.49
EAGLE-3	8B	1.88x	2.40x [2.33, 2.46]	1.28x	5.15/4.33
EAGLE-3	14B	2.39x	2.62x [2.56, 2.69]	1.10x	5.07/4.01

RelaySpec is faster than plain autoregressive decoding in all four pairs, and faster than optimized source reuse in all four, while running with the source model deleted rather than resident. Where a public target-specific proposer exists, RelaySpec retains 90.3% of it for DFlash at 8B, 90.2% for EAGLE-3 at 8B, and 89.1% for EAGLE-3 at 14B, all against the same plain-autoregressive baseline. The margin over source reuse follows the removable share exactly as Section 4 predicts: DFlash at 8B removes 38.8% of request time and gives the largest margin, EAGLE-3 at 14B removes 27.1% and gives the smallest. Figure 3 places all four against every baseline.

What it costs compared to training a new proposer. Fitting one map processes 4,096 examples once. DFlash’s own paper reports training its drafter on about 800,000 examples for 6 epochs, which is 4.8 million example-passes, roughly 1,172 times more. EAGLE-3 reports about 532,000 examples,

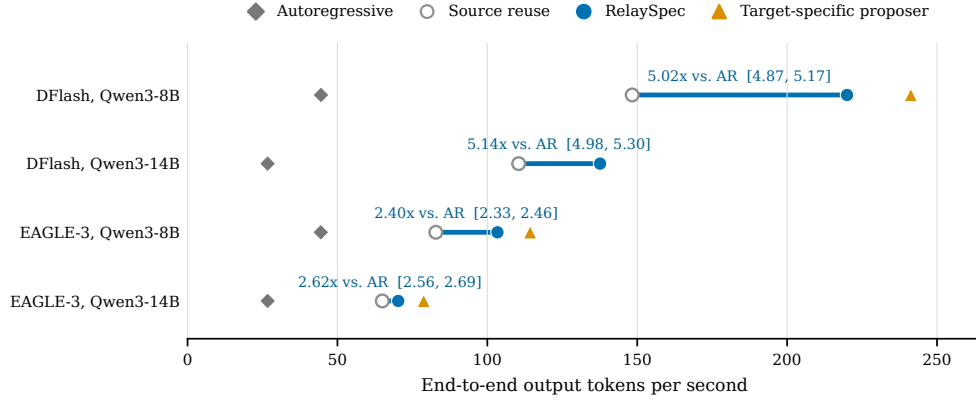


Figure 3: MATH-500 output tokens per second. RelaySpec sits between optimized source reuse and a proposer trained for that exact target, and well above plain autoregressive decoding.

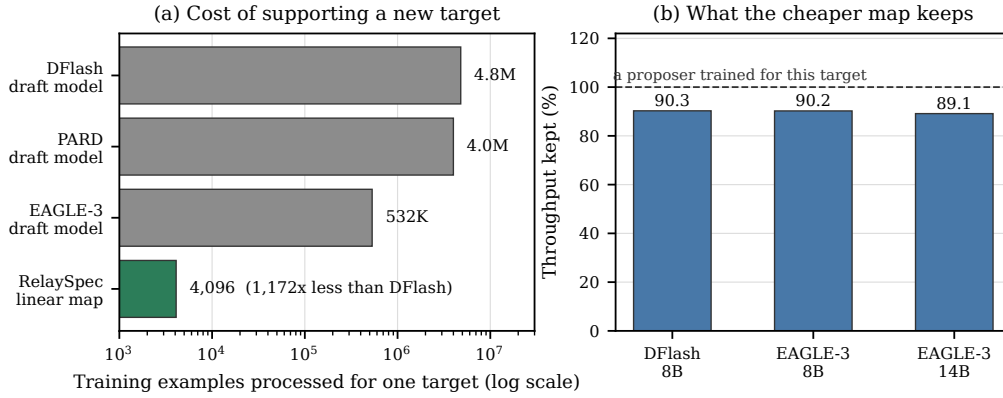


Figure 4: (a) Examples processed to support one new target model, log scale. Counts are epochs times dataset size as reported by each paper. (b) Throughput RelaySpec keeps relative to a proposer trained for that exact target, where a public checkpoint exists for the comparison.

and PARD (An et al., 2026a) reports 8 MI250X GPUs for 4 epochs over roughly one million examples. None of these three papers report GPU-hours for their own training, so we compare data processed, which they do report. Figure 4 puts that cost next to what the cheaper option gives up: 89 to 90 percent of the throughput a proposer trained for that exact target reaches.

Table 3 gives the rest of the deployment cost. Fitting takes 86 to 119 seconds on four GPUs, the map is 52 to 66 million parameters, and deleting the source model saves 7.6 to 7.8 GiB of peak GPU memory, which is 20 to 31 percent.

Table 3: Deployment cost of one map. Fit time is wall clock on four GPUs for DFlash and EAGLE-3 respectively. Memory saving is what deleting the source model returns, measured in isolated processes.

Target	Relay parameters millions	Fit time, DFlash/EAGLE-3 seconds	Peak GPU memory saved GiB / %
8B	52.4	85.6/86.7	7.80 / 30.5
14B	65.5	118.0/118.5	7.63 / 20.3

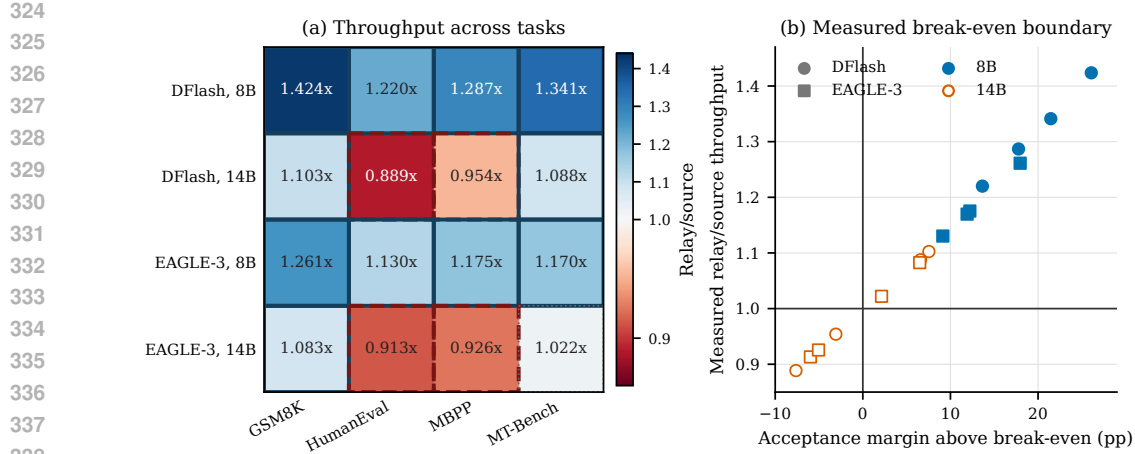


Figure 5: Holding one math-fitted map fixed across four unseen tasks. Left: paired throughput ratio against optimized source reuse per task, the harder of the two baselines, solid border when the interval is fully above one. This sweep was run without a plain-autoregressive arm, so unlike every other result in this paper it is reported against source reuse. Table 2 is where the plain-AR comparison is measured. Right: measured accepted token ratio minus the break-even ratio of Equation 3, so points right of zero are predicted faster and are faster, which holds in every cell.

Beyond math. The map is fitted once, on math prompts only, and we then hold it fixed across GSM8K, HumanEval, MBPP and MT-Bench, four tasks it never saw during fitting. It generalizes: with the source model deleted it keeps 61.5 to 85.4 percent of the accepted tokens per cycle source reuse achieves across all 16 task settings, and quality is unaffected: source reuse and RelaySpec agree on the official score for all 4,680 scored generation pairs across MATH, GSM8K, HumanEval and MBPP. Equation 3 also earns its keep here, predicting the correct speed direction in 16 of 16 settings before any speed is compared, with 0.34% mean absolute error (Figure 5). Appendix D reports every task setting individually.

Appendix F checks the fitting recipe itself: a closed-form solution of the same objective and a nonlinear map both underperform the plain linear map fitted by gradient descent, and fit quality is still improving at 8,192 examples.

7 HOW FAR CAN ONE FROZEN PROPOSER BE MOVED?

The results above change only the target’s size inside one family. We now change the target in four harder ways and measure each against plain autoregressive decoding on 128 MATH-500 prompts, with the same paired bootstrap used for Table 2. Table 4 and Figure 6 report all four.

Table 4: Retargeting a frozen proposer under four changes to the target, 128 MATH-500 prompts each. Speedups are paired against plain autoregressive decoding measured in the same run, exactly as in Table 2.

Change to the target	RelaySpec over plain AR [95% CI]	Accepted tokens per cycle
Independently fine-tuned descendant of the target	4.92x [4.68, 5.18]	6.89
Frozen decoder plus an adapter, 8B map serving a 14B target	4.12x [3.89, 4.35]	5.95
Different model family, and a smaller target than the source	2.45x [2.13, 2.80]	4.05
Different tokenizer, bridged at text granularity	2.26x [2.00, 2.56]	3.21

An independently fine-tuned descendant needs no refitting. We applied the map fitted for base Qwen3-8B, unchanged, to `nvidia/Nemotron-Orchestrator-8B`, a publicly released model

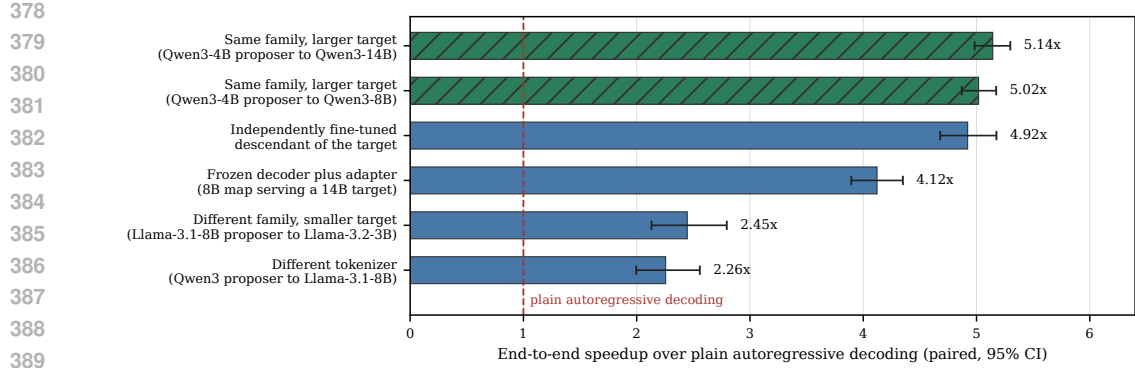


Figure 6: Every retargeting setting, measured against plain autoregressive decoding with paired 95% intervals. Hatched rows are the same-family main results of Table 2, plain rows the four harder target changes of Table 4. All sit well above the dashed no-proposer line.

fine-tuned from Qwen3-8B by a different group. It reaches 4.92x plain autoregressive decoding with no refitting of any kind, close to the 5.02x the same map reaches on the base model it was fitted for, and accepts 6.89 tokens per cycle against 6.98 on the base. The reason it survives is that R is fitted against a basis, not against a behaviour: it only has to send the target’s tapped directions to the directions the proposer expects, and the normalization downstream of it discards everything except direction. Instruction fine-tuning changes where a model goes in that space far more than it rebuilds the space itself, so the map keeps applying. This matters in practice because deployed models are usually fine-tuned descendants of a released base rather than the base itself.

An already-fitted decoder can serve a larger target through an adapter alone. Two maps fitted independently, one for Qwen3-8B and one for Qwen3-14B, produce closely aligned outputs on the same text: cosine similarity 0.973, linear CKA 0.732, and 0.897 overlap between their top-32 output directions. That alignment suggests the decoder half of a fitted map is reusable. Freezing the Qwen3-8B map entirely and training only a small adapter that feeds it Qwen3-14B’s hidden states reaches 4.12x, so a deployment that already holds one fitted map can extend to a larger target in the same family without refitting the map itself.

A different model family works, and the target can be smaller than the source. We repeated the method inside the Llama family, with no Qwen3 model involved anywhere: a released DFlash proposer trained for Llama-3.1-8B, fitted onto the smaller Llama-3.2-3B, reaches 2.45x. Two things follow. The method is not specific to Qwen3, and it runs in both size directions, since our Qwen3 results fit a 4B source onto larger 8B and 14B targets while this one fits an 8B source onto a smaller 3B target.

Even a tokenizer boundary can be bridged. When proposer and target use different vocabularies they cannot agree on what a token is, so the acceptance check has to move to text: we decode each proposed block, re-encode it with the target’s tokenizer, and let the target verify and commit there. Fitting needs a matching change, because the two models no longer share a token sequence, so we align supervision by character span rather than token position, pairing each target position with the source position covering the same underlying text. Applied to a Qwen3-trained DFlash proposer retargeted onto Llama-3.1-8B, this reaches 2.26x with 3.21 accepted tokens per cycle. Appendix I gives the procedure and the one caveat this setting carries, which is that text-granularity verification agrees with plain decoding on 25 percent of prompts rather than nearly all of them.

What this says about the method. One fitted map is not tied to the single target it was fitted for. It carries to a fine-tuned descendant of that target unchanged, to a larger target in the same family through an adapter over a frozen decoder, into an untrained model family, onto a target smaller than the proposer’s own source, and across a tokenizer boundary. Every one of these remains substantially faster than plain autoregressive decoding, which is the comparison a deployment actually faces when no target-specific proposer exists.

8 RELATED WORK

DFlash and EAGLE-3 train proposers that read the target’s hidden states (Chen et al., 2026; Li et al., 2025). DFlare scales draft capacity (Zhang et al., 2026a) and RepSpec merges extra capacity away before inference (Huo et al., 2026). These improve generation for the model the proposer trained with. We study what happens afterwards, when the proposer is fixed and the target changes.

A second line avoids the coupling by design. PARD serves a whole family of sizes with one drafter (An et al., 2026a;b), OmniDraft adapts a drafter online across families and vocabularies (Ramakrishnan et al., 2025), and SD² steers a pretrained drafter with injected features and then fine-tunes the drafter itself on top, since their own ablation shows a frozen drafter underperforms (Berdoz et al., 2026). Others adapt the draft model’s weights directly: EDA’s shared and private split (Lin et al., 2026a), HyperDFlash’s retraining (Lin et al., 2026b), and VSD’s training for expected accepted length (Zou et al., 2026). A third line keeps the draft architecture fixed and trains it against the target’s own feedback (Lei et al., 2026; Qian et al., 2026; Zhang et al., 2026b). Every one of these methods changes the proposer’s weights to serve a new target. RelaySpec is the only one that does not.

Table 5 lists the methods that publish both a headline speedup over plain autoregressive decoding and an adaptation cost, quoting each exactly as published next to the hardware and engine it was measured on. We do not treat the speedup column as comparable across rows, including ours: Abramovich et al. (2026) report that engine and hardware change measured speedup substantially, and each row here was measured on a different combination of the two. The training-data column is the column that is comparable, because it is what each paper reports needing to support one new target and it does not depend on hardware.

Table 5: Methods that publish both a speedup over plain autoregressive decoding and an adaptation cost, quoted as published. RelaySpec is the only row that leaves the proposer’s weights untouched, and the only one whose adaptation cost is a number of examples rather than a training run. Methods that change the proposer but publish no comparable cost are in the text.

Method	Changes proposer?	Reported speedup over plain AR, on its own hardware and engine (not comparable across rows)	Training data for one new target
DFlash	Trains from scratch	4.86x (H200, Transformers)	800K examples, 6 epochs
EAGLE-3	Trains from scratch	4.40x (varies, vLLM/SGLang)	532K examples
PARD, PARD-2	Yes	3.67x (A100-40GB, vLLM)	About 1M examples
SD ²	Yes, steered	1.20x average tokens/s (A100-80GB)	5 tasks, 96 examples each
RelaySpec (this work)	No, frozen	5.02x (RTX 6000 Ada, Transformers)	4,096 unlabeled examples

The comparison we can make cleanly is adaptation cost against throughput kept, since we are the only row that reuses an existing proposer rather than training one: Figure 4 reports 89 to 91 percent kept at roughly three orders of magnitude less data. Closest in mechanism, though not in purpose, are methods that map states between models to speed up first-pass processing rather than the decoding loop: KVShot reuses the target’s own cache for longer drafts (Liu et al., 2026), and cross-model cache transfer learns a linear map onto another model’s prefill cache (Heo et al., 2026).

9 LIMITATIONS AND CONCLUSION

We test greedy decoding, one request at a time, BF16, one fixed seed, and a Transformers runtime, so we measure neither seed variance nor concurrent-request behaviour and these are not production serving numbers (Abramovich et al., 2026). Figure 4 compares examples processed rather than GPU-hours, which the papers involved do not publish. The cross-tokenizer setting verifies at text rather than token granularity, giving up exact agreement with plain decoding (Appendix I), and fitting still needs the source model once, though generation never uses it again.

A proposer that reads a model’s internal states need not stay tied to it. The slot it reads through is one linear layer, and refitting that layer on 4,096 unlabeled examples repoints it at a new target, deletes the source model, and keeps working across fine-tuned descendants, sizes, families and tokenizers. Retargeting a frozen proposer is a fitting problem over one linear interface, not a retraining problem.

AI USE STATEMENT

In this work, we used generative AI tools to help refine hypotheses, design experiments, implement and test code, check the runtime analysis, interpret logged results, summarize related literature, and draft and edit this paper. The authors checked every cited claim against its primary source, inspected the raw experimental artifacts, ran the automated test suite, and reviewed all AI-assisted text and code by hand, taking full responsibility for the paper’s final content.

REPRODUCIBILITY STATEMENT

Appendix C gives every fitting, hardware, generation, metric, and uncertainty setting used. Appendices D through I give the full result matrix, design checks, and the cross-family procedure. Our supplement contains exact model identifiers, configuration files, prompt manifests, raw timing records, scoring code, and a script that regenerates every number and figure.

REFERENCES

- Talor Abramovich, Maor Ashkenazi, Carl Putterman, Benjamin Chislett, Tiyasa Mitra, Bitu Darvish Rouhani, Ran Zilberstein, and Yonatan Geifman. SPEED-Bench: A unified and diverse benchmark for speculative decoding. In *43rd International Conference on Machine Learning*, 2026. URL <https://arxiv.org/abs/2604.09557>. Accepted paper.
- Zihao An, Huajun Bai, Ziqiong Liu, Dong Li, and Emad Barsoum. PARD: Accelerating LLM inference with low-cost PARallel draft model adaptation. In *International Conference on Learning Representations*, 2026a. URL <https://openreview.net/forum?id=XbOyv7iVGL>.
- Zihao An, Taichi Liu, Ziqiong Liu, Dong Li, Ruofeng Liu, and Emad Barsoum. PARD-2: Target-aligned parallel draft model for dual-mode speculative decoding. *arXiv preprint arXiv:2605.08632*, 2026b. URL <https://arxiv.org/abs/2605.08632>.
- Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021. URL <https://arxiv.org/abs/2108.07732>.
- Frédéric Berdoz, Peer Rheinboldt, and Roger Wattenhofer. Steering pretrained drafters during speculative decoding. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pp. 30067–30075, 2026. doi: 10.1609/aaai.v40i36.40255. URL <https://ojs.aaai.org/index.php/AAAI/article/view/40255>.
- Jian Chen, Yesheng Liang, and Zhijian Liu. DFlash: Block diffusion for flash speculative decoding. In *43rd International Conference on Machine Learning*, 2026. URL <https://arxiv.org/abs/2602.06036>. Accepted paper.
- Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Joshua Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021. URL <https://arxiv.org/abs/2107.03374>.
- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021. URL <https://arxiv.org/abs/2110.14168>.

-
- Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset. In *Advances in Neural Information Processing Systems 34, Datasets and Benchmarks Track*, 2021. URL <https://openreview.net/forum?id=7Bywt2mQsCe>.
- Taekyung Heo, Rasoul Shafipour, Ritchie Zhao, Maximilian Golub, Mohammad Mahdi Kamani, Ritika Borkar, Makesh Tarun Chandran, Pantea Zardoshti, and Bitu Darvish Rouhani. Cross-model KV cache transfer in LLM families: A closed-form linear mapping for prefill reuse. *arXiv preprint arXiv:2608.03893*, 2026. URL <https://arxiv.org/abs/2608.03893>.
- Feiye Huo, Jianchao Tan, Jiahao Liu, Zixu Jiang, Jiacheng Li, Jingang Wang, Xunliang Cai, and Shengli Sun. RepSpec: Structural re-parameterized draft model training for speculative decoding. In *International Conference on Learning Representations*, 2026. URL https://proceedings.iclr.cc/paper_files/paper/2026/hash/d70ea003729b440b89a2f958a5554c1f-Abstract-Conference.html.
- Haodi Lei, Yafu Li, Haoran Zhang, Shunkai Zhang, Qianjia Cheng, Xiaoye Qu, Ganqu Cui, Bowen Zhou, Ning Ding, Yun Luo, and Yu Cheng. Draft-OPD: On-policy distillation for speculative draft models. *arXiv preprint arXiv:2605.29343*, 2026. URL <https://arxiv.org/abs/2605.29343>.
- Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. EAGLE-3: Scaling up inference acceleration of large language models via training-time test. In *Advances in Neural Information Processing Systems 38*, 2025. URL https://proceedings.neurips.cc/paper_files/paper/2025/hash/c7b5a35ea98b62512a869c19ea7b03cb-Abstract-Conference.html.
- Luxi Lin, Zhihang Lin, Zhanpeng Zeng, Yuhao Chen, Qingyu Zhang, Jixiang Luo, Xuelong Li, and Rongrong Ji. Efficiently aligning draft models via parameter- and data-efficient adaptation. *arXiv preprint arXiv:2603.09527*, 2026a. URL <https://arxiv.org/abs/2603.09527>.
- Luxi Lin, Shuang Peng, Rui Ma, Junhao Hua, Shuwei Fan, Zhengda Qin, Qiang Wang, Hongjian Sun, Fangmin Chen, and Songwei Liu. HyperDFlash: Hyper-connection-aligned block speculative decoding with gated residual reduction. *arXiv preprint arXiv:2606.26744*, 2026b. URL <https://arxiv.org/abs/2606.26744>.
- Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. Is your code generated by ChatGPT really correct? rigorous evaluation of large language models for code generation. In *Advances in Neural Information Processing Systems 36*, 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/43e9d647ccd3e4b7b5baab53f0368686-Abstract-Conference.html.
- Tianyu Liu, Yuhao Shen, Xinyi Hu, Baolin Zhang, Hengxin Zhang, Jun Dai, Jun Zhang, Shuang Ge, Lei Chen, Yue Li, and MingCheng Wan. When hidden states drift: Can KV caches rescue long-range speculative decoding? *arXiv preprint arXiv:2604.26412*, 2026. URL <https://arxiv.org/abs/2604.26412>.
- Yu-Yang Qian, Hao-Cong Wu, Yichao Fu, Hao Zhang, and Peng Zhao. When drafts evolve: Speculative decoding meets online learning. In *International Conference on Machine Learning*, 2026. URL <https://arxiv.org/abs/2603.12617>.
- Ramchalam Kinattinkara Ramakrishnan, Zhaocong Yuan, Shaojie Zhuo, Chen Feng, Yicheng Lin, Chenzheng Su, and Xiaopeng Zhang. OmniDraft: A cross-vocabulary, online adaptive drafter for on-device speculative decoding. In *Advances in Neural Information Processing Systems*, 2025. URL https://proceedings.neurips.cc/paper_files/paper/2025/hash/3c2fe1417eed1c6ff9acf169617981ea-Abstract-Conference.html.
- An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025. URL <https://arxiv.org/abs/2505.09388>.

-
- Jiebin Zhang, Zhenghan Yu, Song Liu, Eugene J. Yu, Zheng Li, Dawei Zhu, Jiangshan Duo, Weimin Xiong, Yifan Song, Guanghua Yu, Jianchen Zhu, and Sujian Li. DFlare: Scaling up draft capacity for block diffusion speculative decoding. *arXiv preprint arXiv:2606.02091*, 2026a. URL <https://arxiv.org/abs/2606.02091>.
- Jiebin Zhang, Zhenghan Yu, Liang Wang, Nan Yang, Eugene J. Yu, Zheng Li, Yifan Song, Dawei Zhu, Xingxing Zhang, Furu Wei, and Sujian Li. Learning to draft: Adaptive speculative decoding with reinforcement learning. *arXiv preprint arXiv:2603.01639*, 2026b. URL <https://arxiv.org/abs/2603.01639>.
- Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-judge with MT-Bench and chatbot arena. In *Advances in Neural Information Processing Systems 36, Datasets and Benchmarks Track*, 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/91f18a1287b398d378ef22505bf41832-Abstract-Datasets_and_Benchmarks.html.
- Xiandong Zou, Jianshu Li, Jing Huang, and Pan Zhou. Variational speculative decoding: Rethinking draft training from token likelihood to sequence acceptance. *arXiv preprint arXiv:2602.05774*, 2026. URL <https://arxiv.org/abs/2602.05774>.

A COMPLETE METHOD DETAILS AND TENSOR SHAPES

A.1 EXACTLY WHAT SOURCE REUSE COMPUTES

Our optimized source-reuse baseline never runs the source model on a candidate suffix that ends up rejected. Starting from the source model’s own committed cache, it passes each newly committed token through source-model layers 0 through 33, reads out the hidden states at layers [1, 9, 17, 25, 33], and applies the proposer’s own released fusion layer. This is the strongest baseline we know how to build for reusing a proposer with the source model still present, and it is what all speedups in Section 6 are measured against.

A.2 WHY THE LOSS HAS NO SEPARATE COSINE TERM

Equation 1 already normalizes by the target tensor’s own magnitude, so it penalizes direction error at fixed scale. Adding an explicit cosine term changed the 128-step relative loss by less than the run-to-run spread in our design checks, so we left it out.

B GREEDY CORRECTNESS AND FINITE PRECISION

The proposition in Section 3 holds under exact arithmetic. In BF16 the target’s logits for a block of positions and for a single position are computed by different kernels and can differ in the last bits, which changes the committed token only when two logits are nearly tied. We therefore measure text agreement rather than assume it. DFlash matches character for character on all 2,660 MATH, code and chat pairs. EAGLE-3 differs on 49, every one of which is recorded in our audit, and official scores are unchanged in all cases.

C COMPLETE EXPERIMENTAL PROTOCOL

The learning rate 6×10^{-4} of Section 3 was selected on EAGLE-3 and then held fixed for every other fit in this paper, including all DFlash fits, rather than retuned per setting. Fitting uses weight decay 0, gradient clipping 1.0, and one example per GPU per step across four GPUs. Generation uses greedy decoding, temperature 0, a 2,048 token cap, and thinking mode disabled. Speed intervals are paired bootstrap intervals over requests. Hardware is four RTX 6000 Ada GPUs, BF16 throughout, one request at a time.

D FULL WORKLOAD RESULTS

Table 6: Every task setting in the breadth sweep, 128 or more clustered requests each. Speedup is the paired bootstrap ratio against optimized source reuse, the baseline this sweep was run against, and Exact is the percentage of completions matching plain decoding character for character.

Proposer	Target	Task	n /clusters	Source output tok/s	Relay output tok/s	Speedup [95% CI]	Exact %
DFlash	8B	GSM8K	128/128	116.9	166.4	1.424 [1.404, 1.444]	100.0
DFlash	8B	HumanEval	164/164	118.8	145.0	1.220 [1.202, 1.238]	100.0
DFlash	8B	MBPP	378/378	117.8	151.6	1.287 [1.273, 1.300]	100.0
DFlash	8B	MT-Bench	160/80	60.9	81.6	1.341 [1.318, 1.364]	100.0
DFlash	14B	GSM8K	128/128	88.0	97.1	1.103 [1.084, 1.121]	100.0
DFlash	14B	HumanEval	164/164	87.3	77.6	0.889 [0.874, 0.904]	100.0
DFlash	14B	MBPP	378/378	88.0	84.0	0.954 [0.942, 0.966]	100.0
DFlash	14B	MT-Bench	160/80	45.3	49.3	1.088 [1.065, 1.108]	100.0
EAGLE-3	8B	GSM8K	128/128	86.0	108.5	1.261 [1.244, 1.279]	99.2
EAGLE-3	8B	HumanEval	164/164	71.7	81.1	1.130 [1.117, 1.143]	97.0
EAGLE-3	8B	MBPP	378/378	69.9	82.1	1.175 [1.166, 1.185]	97.6
EAGLE-3	8B	MT-Bench	160/80	42.3	49.5	1.170 [1.150, 1.191]	89.4
EAGLE-3	14B	GSM8K	128/128	67.8	73.4	1.083 [1.068, 1.098]	100.0
EAGLE-3	14B	HumanEval	164/164	55.4	50.6	0.913 [0.901, 0.926]	99.4
EAGLE-3	14B	MBPP	378/378	54.5	50.5	0.926 [0.915, 0.936]	98.7
EAGLE-3	14B	MT-Bench	160/80	33.8	34.6	1.022 [1.000, 1.045]	96.9

E ACCEPTANCE AND WHERE THE TIME GOES

Figure 7 shows how often a proposed position at each depth inside a block is still accepted. Acceptance falls with depth for both source reuse and RelaySpec, and RelaySpec’s curve sits below it by a roughly constant factor rather than collapsing at a particular depth. This is why a single accepted-tokens-per-cycle ratio is enough for Equation 3.

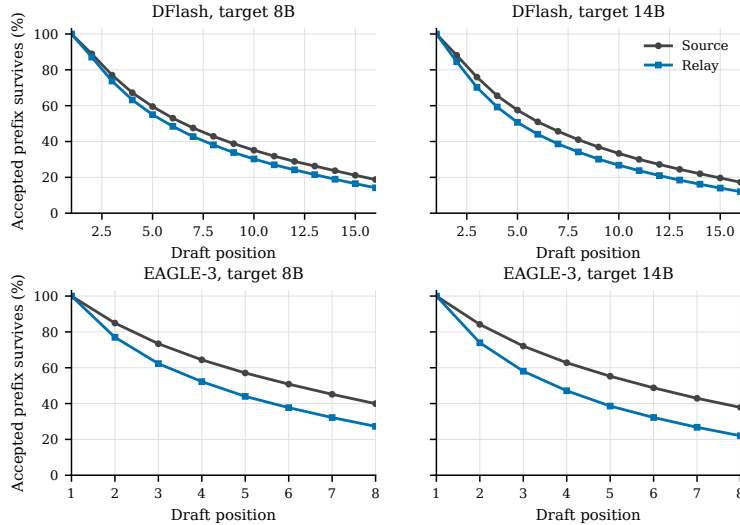


Figure 7: Fraction of proposals still accepted at each position inside a block.

Table 7 gives the exact accepted-tokens-per-cycle ratio behind Figure 5’s margin panel, and Figure 8 shows how source-reuse request time divides between components, whose source-trunk share is what RelaySpec removes.

Table 7: Accepted tokens per cycle for every measured setting.

Proposer	Target	Source accept. tokens/cycle	Relay accept.	Retained	Removed time %
DFlash	8B	7.66	6.98	91.1	38.8
DFlash	14B	7.44	6.49	87.3	30.2
EAGLE-3	8B	5.15	4.33	84.2	33.1
EAGLE-3	14B	5.07	4.01	79.2	27.1

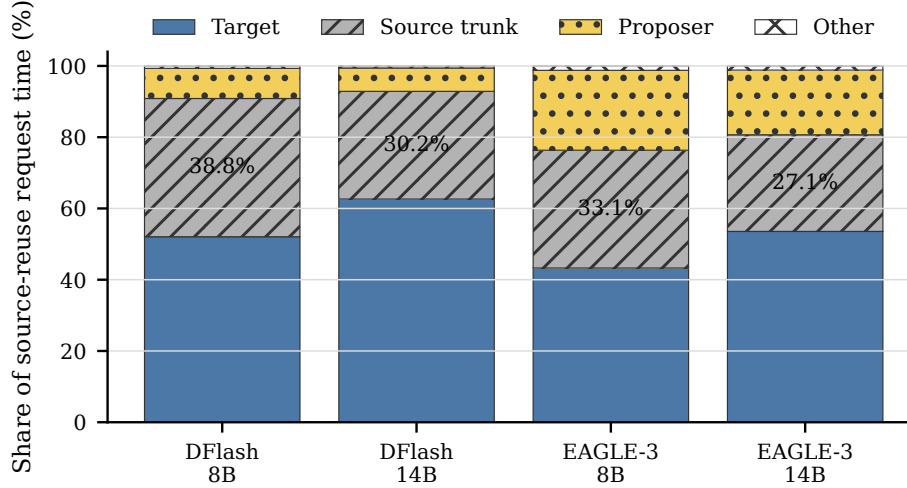


Figure 8: How source-reuse request time divides between components. The labelled source-trunk share is exactly the work RelaySpec removes.

F DESIGN CHECKS THAT DECIDED THE METHOD

Three controlled comparisons fixed the final design. Normalizing the map’s input improved the 128-step relative loss from 0.413 to 0.266 for EAGLE-3 and raised measured speed from 1.035 to 1.237 times, with no change in parameter count or inference cost. A block size of 16 was best for DFlash among 8, 16 and 32. The scale-relative loss in Equation 1 beat a plain squared error at matched steps.

F.1 IS A LINEAR MAP FITTED BY GRADIENT DESCENT THE RIGHT CHOICE?

The main result fits a single linear map by gradient descent on 4,096 unlabeled examples. We check three alternatives directly, all for DFlash on Qwen3-8B, all measured at 128 prompts against the same `optimized_source_reuse` baseline as Table 2, with paired bootstrap 95% intervals, in Table 8.

Table 8: Fitting-design ablations, 128 prompts, MATH-500. Speed is against optimized source reuse. Retention is accepted tokens per cycle relative to the same baseline.

Configuration	Fit examples	Speed	95% interval	Retention
Closed-form, weighted ridge regression	4,096	0.955x	[0.927, 0.982]	52.3%
MLP relay, hidden width 512	4,096	0.872x	[0.841, 0.903]	51.7%
Linear, gradient descent	512	1.150x	[1.118, 1.181]	68.1%
Linear, gradient descent	1,024	1.372x	[1.346, 1.397]	81.1%
Linear, gradient descent	2,048	1.477x	[1.458, 1.496]	87.1%
Linear, gradient descent	8,192	1.573x	[1.558, 1.588]	93.4%

A closed-form solution exists because the loss is linear least squares, provided the normal equations are weighted to match it: gradient descent minimizes Equation 1, which weights each position’s squared error by the inverse energy of its own target vector, so we weight each row by that same factor before accumulating the normal equations, an exact equivalence for a linear map. Solved this way, with no learning rate to tune, it keeps only 52.3 percent of accepted tokens per cycle against 91 percent for gradient descent at the same data budget. It is also not clearly cheaper: collecting the same 4,096 examples’ hidden states takes 221.3 seconds and the linear solve itself 6.8 seconds, comparable to the 86 to 119 seconds gradient descent needs in total, since collecting hidden states dominates either way. Adding a hidden layer (an MLP of width 512 in place of the single linear layer) performs worse still, 51.7 percent retention, the single worst configuration measured anywhere in this paper. Neither alternative improves on the plain linear map. Both are strictly worse uses of the same fitting data. Fit-data size itself has a real, monotonic effect with no measured ceiling in the range we test: retention rises from 68.1 percent at 512 examples to 93.4 percent at 8,192, with no sign of flattening. The paper’s choice of 4,096 examples is therefore a practical operating point inside a still-improving regime, not a demonstrated ceiling on what more fitting data could give. Doubling the fitting data to 8,192 examples raises retention from the 91.1 percent the main 4,096-example map reaches on this pair to 93.4 percent, at roughly twice the fitting time, and every result reported elsewhere in this paper uses the 4,096-example map.

G PROMPT OVERLAP AND TEXT DIFFERENCES

Fitting prompts and evaluation prompts are disjoint by construction. Table 9 reports the closest match between any fitting prompt and any evaluation prompt under a character-level similarity score. The most similar MATH pairs share competition-style templates with different numbers, which limits how strongly MATH alone supports a claim about unrelated tasks. Our code and chat measurements act as the separate check for that.

Table 9: Closest fitting-to-evaluation prompt similarity per task.

Task	Maximum	≥ 0.80	≥ 0.90	≥ 0.95
MATH-500	0.980	47	13	4
GSM8K	0.378	0	0	0
HumanEval	0.290	0	0	0
MBPP	0.333	0	0	0
MT-Bench	0.571	0	0	0

H MEMORY MEASUREMENT

We run source reuse and RelaySpec in two separate processes with the same target, proposer, prompts and settings, record allocated and reserved CUDA memory before any timed request, then reset the peak counters and record peak allocation during timed requests. At 8B peak allocated memory falls from 25.56 to 17.76 GiB. At 14B it falls from 37.57 to 29.94 GiB, both shown in Figure 9.

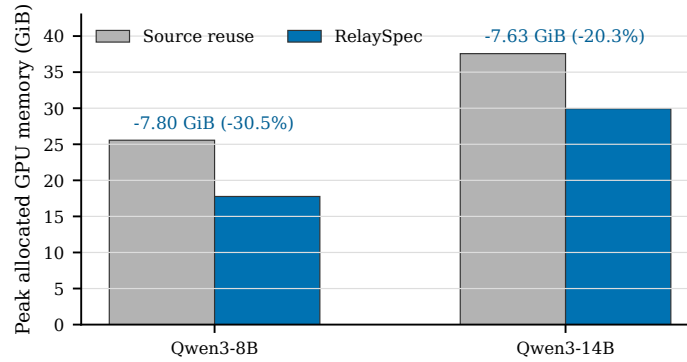


Figure 9: Peak allocated GPU memory in isolated processes.

I CROSS-FAMILY PROCEDURE AND RESULT

When proposer and target do not share a tokenizer, three separate mappings break. The proposer emits token IDs in its own vocabulary, it embeds the tokens it is growing through its own embedding table, and it conditions on the committed prefix. We handle all three by moving agreement to text, which needs no new learned parameters. Each cycle, we decode the proposed block with the source tokenizer, re-encode that text with the target tokenizer, let the target verify and commit in its own vocabulary, then decode what was committed and re-encode it back into the source vocabulary to continue proposing. Correctness carries over because the target still produces every committed token, now at text granularity rather than token granularity. Acceptance gets coarser, because a mismatch inside a token ends the rest of the block.

Fitting needs a second change. The usual loss compares the source and target hidden states position by position, which assumes one shared token sequence. With two tokenizers there is no general position-by-position correspondence between sequences of different lengths.

Aligning supervision by character span. We recover real per-position supervision without assuming a shared token sequence by aligning on the underlying text instead of on token position. For each fitting example, we identify the character windows in the rendered prompt where the actual problem and solution content appears, excluding chat-template boilerplate that has no correspondent on the other side. For each target position, we find the source position whose character span covers the same underlying text and pair those two positions for the loss in Equation 1, exactly as same-tokenizer fitting pairs positions by index. This needs no new learned parameters and applies only at fitting time. The inference-time procedure above is unchanged.

Fitted this way and measured at 128 prompts under the same paired protocol as Table 2, the frozen Qwen3-trained proposer reaches 3.208 accepted tokens per cycle on Llama-3.1-8B and 2.26x plain autoregressive decoding, the number reported in Table 4. Fitting takes 94.5 seconds on four GPUs, inside the 86 to 119 second range Section 3 reports for same-tokenizer fits. The committed text matches plain decoding on 25 percent of prompts, the one caveat this setting carries.

The exact-match gap, traced rather than left unexplained. We traced a specific disagreement directly: on one held-out prompt, greedy autoregressive decoding and the cross-family relay agree for the first 34 tokens and then diverge. At the divergence point the logit gap between the two candidate tokens is zero to reported precision, a genuine near-tie rather than a large disagreement. This is the same block-versus-single-position floating-point kernel signature Appendix B already documents for same-tokenizer verification, occurring far more often here because cross-family verification runs at text granularity, so a mismatch anywhere inside a proposed token’s re-encoding can trigger a tie-break that a token-granularity check would not see in the same way. We do not claim every one of the 75 percent of disagreements traces to exactly this mechanism, only that the one we traced does, and that it is the same known phenomenon rather than a new, unexplained failure mode.